Detailed Project Report (DPR)

Vehicle Rental Management System — RentRide

Field	Details
Project Title	Vehicle Rental Management System
Product Name	RentRide
Program / Mentor	Unified Mentor (RentiGo brief)
Project Type	Full-stack web application
Document Type	Detailed Project Report (Submission)
Version	1.0
Date	July 2026

Notice

This project is proprietary. Source code is intended for a **private** GitHub repository. Public cloning or redistribution is not authorized.

RentRide is a centralized, web-based Vehicle Rental Management System that enables customers to rent two-wheelers (2W) and four-wheelers (4W) on a **daily, weekly, or monthly** basis, while allowing rental agencies to manage fleet availability, pricing, and bookings, and administrators to oversee users, listings, pricing categories, and platform analytics.

The system digitizes traditional rental workflows that previously depended on phone calls and spreadsheets. It provides real-time availability checks, conflict-aware booking, role-based dashboards, and KPI analytics — aligning with the Unified Mentor / RentiGo project requirements.

---

2. Problem Statement

Traditional vehicle rental operations rely heavily on manual processes, phone calls, or basic spreadsheets, leading to:

- Vehicle availability conflicts
- Lack of real-time booking updates
- Inconsistent pricing management
- Poor customer experience
- Inefficient fleet utilization

RentRide addresses these issues through a single responsive web platform with authenticated roles (Customer, Agency, Admin), REST APIs, and a PostgreSQL-backed data model.

---

3. Objectives

3.1 Primary Objectives

Objective	How RentRide Achieves It
Digitize the vehicle rental booking process	End-to-end web flow: browse → filter → detail → checkout → booking request
Provide real-time vehicle availability tracking	Vehicle status (`available` / `rented` / `maintenance` / `pending_approval`) + booking conflict checks
Enable flexible rental durations	Daily / weekly / monthly pricing and duration selection
Improve operational efficiency for agencies	Agency dashboard: fleet CRUD, approve/reject bookings, maintenance status
Improve customer booking experience	Search/filters, vehicle details, date/time selection, booking history

3.2 Secondary Objectives

Objective	Implementation
Analytics on vehicle usage and revenue	Admin analytics KPIs (conversion, utilization, revenue, MAU, avg duration)
Scalable expansion across cities	Location field on vehicles; multi-city seed data (Mumbai, Bangalore, Delhi, etc.)

---

4. Scope of Work

4.1 In Scope (Implemented)

- Responsive web application (desktop + mobile layouts)
- Two-wheeler and four-wheeler rentals
- Booking and availability management (including overlap conflict detection)
- Pricing management (per-vehicle daily/weekly/monthly + admin pricing categories)
- Role-based access: Customer, Agency, Admin
- JWT authentication with HTTP-only cookies

4.2 Out of Scope (Phase 1 — as per brief)

- Native mobile applications (Android / iOS)
- GPS tracking of vehicles
- Insurance claim management
- Online payment gateway & refunds (future enhancement)

---

5. System Overview

5.1 Product Positioning

RentRide acts as a marketplace-style rental platform:

1. **Customers** discover and request vehicles.
2. **Agencies** own and operate fleets, approve bookings, and mark maintenance.
3. **Admins** govern users, approve listings, manage pricing templates, and monitor KPIs.

5.2 High-Level Architecture

```
■■■■■■■■■■■■■■■■■■■■■■■■ REST / JSON ■■■■■■■■■■■■■■■■■■■■■■■■ ■ Frontend (React) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ API (Express.js) ■ ■ Vite · Port 3000 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Port 3001 ■ ■ Tailwind CSS v4 ■ JWT cookie session ■
Role-based routes ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ▼
■■■■■■■■■■■■■■■■■■■■■■■■ ■ PostgreSQL ■ ■ Drizzle ORM ■ ■■■■■■■■■■■■■■■■■■■■■■■■
```

Request path (example):

```
VehiclesPage.jsx → api-client.js → GET /api/vehicles → vehicles.js → PostgreSQL → JSON
response
```

---

6. Technology Stack

Layer	Technology	Purpose
Frontend	React 19, JavaScript, Vite	SPA UI
Routing	React Router DOM	Client-side navigation
Styling	Tailwind CSS v4	Responsive, mobile-first UI
Data fetching	TanStack React Query	Caching & async UI state
Backend	Node.js, Express.js	REST API

Database	PostgreSQL	Persistent storage
ORM	Drizzle ORM	Schema, queries, migrations
Auth	JWT + HTTP-only cookies + bcrypt	Secure login / RBAC
Tooling	npm workspaces, concurrently	Monorepo `frontend` + `api`
Deployment targets	Vercel / Netlify (frontend), Node host / Railway / Render (API), Neon / Supabase (DB)	Production-ready options

---

7. Functional Requirements & Implementation Status

7.1 Customer Features

Requirement	Status	Implementation Notes
User registration & login	■	`/login` — customer / agency signup; JWT cookie session
Browse available vehicles	■	`/vehicles` — approved & available fleet
Real-time availability by date	■	Trip dates exclude vehicles already booked for that window
Filter by type (2W / 4W)	■	Bike / Car toggles
Filter by price range	■	Min/max sliders by rental plan
Filter by rental duration	■	Daily / weekly / monthly
Filter by fuel type	■	Petrol, diesel, electric, CNG
Search by brand/model	■	Debounced text search
Location filter	■	Case-insensitive city match
View vehicle details	■	`/vehicles/:id` — photos, specs, pricing
Select rental period (date & time)	■	Checkout with start/end date & time slots
Book vehicle	■	Creates `pending` booking; conflict check

View booking history & status	■	`/bookings` — cancel pending/approved

7.2 Rental Agency Features

Requirement	Status	Implementation Notes
Fleet dashboard	■	`/agency` — stats + fleet + booking requests
Add / edit vehicles	■	Vehicle number, brand, model, pricing, images, location
Rental pricing (daily/weekly/monthly)	■	Stored per vehicle
Availability status	■	available / rented / maintenance / pending_approval
Approve / reject bookings	■	Status transitions on agency bookings
Block for maintenance	■	Status update on fleet vehicles
Track active rentals	■	Dashboard counters & booking list
Agency account activation	■	New agencies start inactive until admin activates

7.3 Admin Features

Requirement	Status	Implementation Notes
Manage users & agencies	■	Activate / deactivate / delete (with safeguards)
Approve vehicle listings	■	Pending vehicles queue
Monitor bookings	■	Platform-wide bookings table
Manage pricing categories	■	Create / update category rate templates
View system analytics	■	KPI dashboard (users, conversion, utilization, revenue, MAU)

---

8. Non-Functional Requirements

Area	Target (Brief)	Approach in RentRide
Performance	Page load < 3s; concurrent bookings	Vite SPA + REST API; indexed booking/vehicle columns; conflict check before insert
Security	Secure auth + RBAC	bcrypt password hashing; JWT in HTTP-only cookies; role checks on protected routes
Usability	Simple, intuitive, mobile-first	Responsive Tailwind layouts; sticky header; clear role dashboards
Scalability	Multi-location & fleet growth	PostgreSQL; location-aware inventory; modular Express routes

---

9. User Roles & Access Control

Role	Access
Customer	Browse, book, view/cancel own bookings
Agency	Manage own fleet, approve/reject bookings for owned vehicles
Admin	Users, listing approval, pricing categories, analytics, all bookings

Unauthorized or wrong-role access returns `401` / `403` from the API.

---

10. User Flow (High-Level)

```
1. User visits RentRide website (Home) 2. Registers / logs in (Customer or Agency) 3. Searches / filters available vehicles 4. Opens vehicle details and selects rental dates & times 5. Submits booking request (status: pending) 6. Agency (or Admin) approves or rejects 7. Customer sees updated status in My Bookings 8. On approval/active, vehicle status moves toward rented; on cancel/complete/reject, availability is restored when safe
```

10.1 Key Screens

Route	Screen
`/`	Landing / search hero
`/login`	Sign in / Sign up

`/vehicles`	Browse + filters
`/vehicles/:id`	Vehicle details
`/checkout`	Confirm booking
`/bookings`	Customer / role booking list
`/agency`	Agency dashboard
`/admin`	Admin console

---

11. Data Model

11.1 Core Entities

Entity	Description
User	Customers, agencies, admins (role, active flag, agency name)
Vehicle	Fleet listing with specs, pricing, status, location, owner
Booking	Rental request lifecycle with dates, times, price, status
PricingCategory	Admin-defined rate templates by vehicle type
BookingConflict	Log of booking attempts rejected for a date clash; feeds the conflict-rate KPI
Location	Represented as a city/location string on each vehicle (multi-city support) — see 11.5

11.2 Entity Relationship (Logical)

```
User (1) ■■■■■■■■■ (N) Vehicle User (1) ■■■■■■■■■ (N) Booking [as customer] Vehicle (1) ■■■■■■ (N) Booking Admin manages PricingCategory (independent catalog)
```

11.3 Sample Vehicle Attributes

- Vehicle name, number, brand, model
- Type (2W / 4W)
- Model year, fuel type, transmission, seats, mileage
- Images & description
- Rental price: daily / weekly / monthly
- Availability / approval status

- Location (city)

11.4 Booking Status Lifecycle

`pending` → `approved` / `rejected` / `cancelled` → `active` → `completed`

Conflict detection blocks overlapping bookings for the same vehicle when status is `pending`, `approved`, or `active`.

The same overlap rule drives browse-time availability: when a customer selects trip dates, `GET /api/vehicles?from=&to=` excludes any vehicle already committed for that window, so the list a customer sees matches exactly what booking creation will accept.

11.5 Location Modelling — Design Decision

`Location` is stored as a city string on each `Vehicle` rather than as a separate table with a foreign key.

Rationale. Phase 1 targets a single-city pilot with expansion planned later. The city field already satisfies every location requirement in scope — customers filter by city (case-insensitive match), agencies set a city per vehicle, and vehicles from multiple cities coexist in one fleet. A separate table would add a join, a migration and a managed-city admin screen without changing any user-visible behaviour at this stage.

Trade-off accepted. City names are not constrained to a controlled vocabulary, so two spellings of the same city are two values.

Normalisation path. When per-city rules arrive — city-specific pricing, taxes, pickup hubs, or a city-level admin — the migration is: create `Location (id, city, state, pincode)`, seed one row per distinct existing city, add `Vehicle.locationId` as a foreign key, backfill it from the current text, then drop the text column. Filtering moves from a string match to a join. No booking or pricing logic is affected.

---

12. API Design (REST)

Area	Endpoints (summary)
Auth	`POST /api/auth/login`, `register`, `logout`; `GET /api/auth/me`
Vehicles	`GET /api/vehicles`, `GET /:id`, `POST /`, `PUT /:id`, `DELETE /:id`
Bookings	`GET /api/bookings`, `POST /`, `PATCH /:id`
Agency	`GET /api/agency/dashboard`
Pricing	`GET /api/pricing` (public categories)

Admin	Users, vehicles approval, bookings, pricing CRUD, analytics
Health	`GET /api/health`

All successful responses follow: `{ success: true, data: ... }`
Errors follow: `{ success: false, error: "message" }`

---

13. Key Performance Indicators (KPIs)

Tracked on the **Admin Analytics** dashboard:

KPI	Definition (as implemented)
Registered users	Count of customer accounts
Booking conversion rate	Approved ÷ non-cancelled bookings
Vehicle utilization rate	Rented vehicles ÷ total vehicles
Average rental duration	Average `totalDays` of active-class bookings
Monthly active users	Unique customers active in last 30 days (bookings + new customers)
Total revenue	Sum of prices for approved/active/completed bookings
Booking conflict rate	Rejected-for-clash attempts ÷ (those + bookings created), from the `BookingConflict` log
Pending approvals	Vehicles awaiting admin listing approval

---

14. Project Structure

```
RentRide/ ■■■ frontend/ # React + Vite UI (port 3000) ■ ■■■ src/pages/ # Home, Vehicles, Checkout, Bookings, Agency, Admin, Login ■ ■■■ src/components/ # Header, Footer, VehicleCard, AuthProvider, etc. ■ ■■■ src/lib/ # api-client, adapters, filters ■■■ api/ # Express REST API (port 3001) ■ ■■■ src/routes/ # auth, vehicles, bookings, agency, admin, pricing ■ ■■■ src/lib/ # schema, db, auth, booking conflict, serialize ■ ■■■ scripts/seed.js # Demo users & fleet ■■■ docs/ # Project documentation (this DPR) ■■■ package.json # npm workspaces monorepo ■■■ README.md # Setup & run instructions
```

---

15. Setup, Build & Demo

15.1 Prerequisites

- Node.js 18+
- PostgreSQL 14+ (local or cloud: Neon / Supabase)

15.2 Local Run

```
npm install cp .env.example .env # Set DATABASE_URL and JWT_SECRET npm run db:push npm run seed
npm run dev
```

- Frontend: http://localhost:3000
- API: http://localhost:3001/api/*

15.3 Demo Accounts (after seed)

Demo login credentials are stored only in the project owner's local `.env` (`SEED_*` variables) and are **not published** in this repository.

---

16. Assumptions & Constraints

16.1 Assumptions

- Vehicle data is kept accurate by agencies and reviewed by admins.
- Users follow rental policies agreed at booking time.
- Admin approvals for agencies and listings are processed in a timely manner.
- Phase 1 payments are offline / pay-at-pickup (no payment gateway).

16.2 Constraints

- Fixed development timeline for Phase 1.
- Limited budget (no native apps, GPS, or insurance modules).
- No real-time vehicle GPS tracking.
- Online payments & refunds deferred to future phases.

---

17. Deliverables (Submission Checklist)

Deliverable	Status
Functional web application	■ RentRide frontend + API
Admin dashboard	■ `/admin` with users, bookings, pricing, analytics

Agency dashboard	■ `/agency` fleet & booking management
PRD / requirements alignment	■ This DPR + implemented features matrix
Technical documentation	■ Architecture, stack, API, data model (this document)
Deployment-ready build	■ `npm run build` (frontend Vite production build; API Node start)
Seed / demo data	■ `api/scripts/seed.js`

---

18. Testing Summary (Recommended for Evaluation)

Scenario	Expected Result
Customer login	Session cookie set; redirect to browse/bookings
Filter Bike / Car / fuel / city / price	Vehicle list updates correctly
Create booking with overlapping dates	API returns conflict (409)
Agency approve booking	Status updates; vehicle may move to rented
Customer cancel pending booking	Status becomes cancelled
Admin deactivate agency	Agency cannot operate until reactivated
Admin approve pending vehicle	Vehicle becomes available in browse

---

19. Expected Impact

- Improved rental operational efficiency for agencies
- Reduced double-booking / availability conflicts
- Better customer experience through self-serve search and booking
- Increased visibility into revenue and utilization for platform admins
- Foundation for multi-city expansion

---

20. Future Enhancements

Aligned with the project brief:

1. Online payments & refunds

2. Native mobile apps (Android / iOS)
3. GPS vehicle tracking
4. Dynamic pricing
5. Insurance & damage management
6. Richer conflict-rate logging and alerting
7. Email / SMS notifications for booking status changes

---

21. Conclusion

RentRide successfully implements the **Vehicle Rental Management System** requirements defined in the Unified Mentor / RentiGo audit. The platform delivers a complete Phase-1 solution covering customer booking, agency fleet operations, and admin governance on a modern React + Express + PostgreSQL stack, with clear scope boundaries and a roadmap for payments, mobile, and tracking in later phases.

---

22. References

1. Unified Mentor — Vehicle Rental Management System project brief (RentiGo)
2. RentRide source repository — `frontend/`, `api/`, `README.md`
3. Drizzle ORM & PostgreSQL documentation
4. React, Vite, Express.js official documentation

End of Detailed Project Report